90
Pass

Saurav Kumar

PDF generated at: 18 Feb 2026 17:35:05 UTC

View this report on HackerRank

Score

90% • 157.5 / 175

scored in eClerx - Java Coding Test - 3+ Years New in 19 min 24 sec on 18 Feb 2026 22:43:26 IST

Candidate Information

Email	saurav.kumar3279@gmail.com
Test	eClerx - Java Coding Test - 3+ Years New
Candidate Packet	View
Taken on	18 Feb 2026 22:43:26 IST
Time taken	19 min 24 sec/ 90 min
Work Experience	4 years
Invited by	Vipul
Invited on	18 Feb 2026 14:34:23 IST

Skill Distribution

No.	Skill	Score
1	Spring Boot Advanced	33%

2	Spring Boot Intermediate	25%
3	Problem Solving Intermediate	100%
4	Java Intermediate	100%

Tags Distribution

Spring Boot	30%	Hard	33%
TDD	50%	Spring	25%
JPA	50%	Spring Data	50%
Transactional	50%	Medium	95%
Application Properties Annotations	0%	Pointers	100%
Greedy	100%	Java	100%

Questions

Spring Boot (Advanced) • 7.5 / 25

No.	Question	Skill	Score	Code Quality	Time Taken	Status
1	Determine the Output Multiple Choice	Spring Boot (Advanced)	0/5	-	59 sec	⊗

2	Output Multiple Choice	Spring Boot (Advanced)	0/5	-	51 sec	✗
3	Response Multiple Choice	Spring Boot (Advanced)	5/5	-	1 min	✓
4	Spring Data Transactional Multiple Choice	Spring Boot (Intermediate)	2.5/5	-	59 sec	✓
5	Configuration Properties Multiple Choice	Spring Boot (Intermediate)	0/5	-	1 min 30 sec	✗

Problem Solving (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
6	Quiz Competition Coding	Problem Solving (Intermediate)	75/75	-	6 min 17 sec	✓

Java (Intermediate) • 75 / 75

No.	Question	Skill	Score	Code Quality	Time Taken	Status
7	Java: Optimized Sequential Video Watching Coding	Java (Intermediate)	75/75	-	6 min 59 sec	✓

1. Determine the Output

Incorrect

Multiple Choice | Spring Boot | Hard

Question description

SpringQuestionApplication.java

```
package com.hackerrank.spring;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringQuestionsApplication {
   public static void main(String[] args) {
      SpringApplication.run(SpringQuestionsApplication.class, args);
   }
}
```

Lie.java

```
package com.hackerrank;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class Lie implements CommandLineRunner {
   @Override
   public void run(String... args) throws Exception {
      System.out.println("Truth is behind the lies");
      System.out.println("This is working perfectly");
      throw new Exception("Does this work?");
   }
}
```

Truth.java

```
package com.hackerrank.spring.reality;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class Truth implements CommandLineRunner {
   @Override
   public void run(String... args) throws Exception {
```

```
        System.out.println("Truth is in your eyes");
    }
}
```

What will be printed on the console when the "SpringBootApplication" class is executed?

Interviewer guidelines

Correct Answer:

Option [A] – "Truth is in your eyes"

Explanation: Spring Boot scans only the package of the `@SpringBootApplication` class and its subpackages. `Truth` is under a subpackage and runs. `Lie` is in a parent package, so it is **not scanned** and its runner does not execute.

Incorrect Answers:

Option [B] – Multiple prints + exceptions

Explanation: `Lie` never runs, so its prints and exception do not occur.

Option [C] – Prints from Lie only

Explanation: `Truth` is discovered and executed; `Lie` is not.

Option [D] – Prints from Lie, then Truth

Explanation: `Lie` is outside the scan path and never executed.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ Truth is in your eyes ✓
- ○ Truth is in your eyes Truth is behind the lies This is working perfectly And exception occurs and application stops
- ● **Truth is behind the lies This is working perfectly And exception occurs and application stops**

Truth is behind the lies This is working perfectly And exception occurs containing “Does this work?” Truth is in your eyes

No comments.

2. Output

Incorrect

Multiple Choice | Hard | Spring Boot | TDD

Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
   SomeClassToTest someClassToTest;

   @Before
   public void init()
   {
     log = new File("log.txt");
     logWriter = new FileWriter(log);
   }

   @After
   public void teardown()
   {
      log.delete();
   }

   @Test
   public void test1()
```

```
    {
        logWriter.write("Entered Test 1");
        assertTrue(someClassToTest.someMethod());
    }

    @Test
    public void test2()
    {
        logWriter.write("Entered Test 2");
        assertTrue(someClassToTest.otherMethod());
    }
}
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ○ The output file is not created.
- ● **The output file is created but not deleted.**
- ○ The output file is created and eventually deleted. ✓
- ○ The create and delete blocks will not be executed.

No comments.

3. Response

Correct

Multiple Choice | Hard | Spring Boot | TDD

Question description

Which response will cause the following test case to fail?

```
@Test
   @DisplayName("Filter Price Check 3")
   public void FilterPriceCheck3() throws Exception {
      mockMvc.perform(get("/filter/price/150/500"))
            .andDo(print())
            .andExpect(jsonPath("$[*].product", containsInAnyOrder("Product 1", "Product 2" , "Produc
t 5")))
   }
```

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [{"product" : "Product 5"} , {"product" : "Product 1"}, {"product" : "Product 2"}]

- [{"product" : "Product 5"} , {"product" : "Product 2"}, {"product" : "Product 1"}]

- [{"product" : "Product 1"} , {"product" : "Product 2"}, {"product" : "Product 5"}]

[{"product" : "product 5"} , {"product" : "product 2"}, {"product" : "product 1"}]

No comments.

4. Spring Data Transactional

Partially correct

Multiple Choice | Spring | Spring Boot | JPA | Spring Data | Transactional | Medium

Question description

Given the following code, what happens when *StudentRepsitoryTest* runs?

Lesson.java

```
@Entity
public class Lesson {
   @Id
   private Long id;
   private String name;
   private Integer courseLength;
   private Integer courseLevel;
   @JsonBackReference
   @ManyToOne(fetch = FetchType.LAZY)
   @JoinColumn(name = "student_id")
   private Student student;
   private boolean inProgress;
   //getter & setters
}
```

Student.java

```
@Entity
public class Student {
   @Id
   private Long id;
```

```
    private String name;
    private String surname;
    @JsonManagedReference
    @OneToMany(mappedBy = "student", fetch = FetchType.EAGER, cascade = {CascadeType.PERSIST,
CascadeType.REFRESH})
    private List<Lesson> lessons;
    //getter & setters
}
```

StudentRepository.java

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long>{
    //Y
    Optional<Student> findById(Long id);
}
```

LessonRepository.java

```
@Repository
public interface LessonRepository extends JpaRepository<Lesson, Long> {
}
```

StudentService.java

```
@Service
public class StudentService{
    private StudentRepository studentRepository;
    public StudentService(StudentRepository studentRepository, LessonRepository lessonRepository)
{
        this.studentRepository = studentRepository;
        this.lessonRepository = lessonRepository;
    }

    public void save(Student student) {
        studentRepository.save(student);
    }

    //other crud metods
```

```
    //X
    public void deleteStudentWithLessons(Long id) throws SQLException {
        Student student =
                studentRepository.findById(id).orElseThrow(() -> new RuntimeException("Student not foun
d"));
        List<Lesson> lessons = student.getLessons();
        lessons.stream().forEach(lesson -> lesson.setStudent(null));
        studentRepository.delete(student);

        boolean inProgress =lessons.stream()
                .map(Lesson::isInProgress)
                .filter(progress -> progress == true)
                .findFirst()
                .orElseGet(() -> false);
        if (inProgress) {
            throw new SQLException("Cannot delete all Lessons. There are lessons in progress state");
        }
        lessonRepository.deleteAll(lessons);
    }
}
```

StudentRepositoryTest.java

```
@SpringBootTest
class StudentRepositoryTest {
    @Autowired
    StudentRepository studentRepository;
    @Autowired
    StudentService studentService;
    @Autowired
    LessonRepository lessonRepository;
    Student student;
    @BeforeEach
    public void init() {
        student = Student.builder().name("Jorn").surname("Deynhoven").build();
        studentRepository.save(student);
        lessonRepository.saveAll(List.of(
                Lesson.builder().name("Music").courseLength(10).student(student).courseLevel(123).build
(),
                Lesson.builder().name("Art").courseLength(20).courseLevel(123).student(student).build(),
                Lesson.builder().name("Trance").courseLength(30).courseLevel(123).student(student).inPro
gress(true).build()
        ));
```

```
    }
    @Test
    void deleteStudentWithLessonsTest() throws SQLException {
        studentService.deleteStudentWithLessons(student.getId());
    }
}
```

Interviewer guidelines

Candidate has to know how to use spring data in any spring application which uses spring data. If an entity wanted to remove from database without removing the child attributes, then first relation between them should be removed. Without doing this, spring will throw DataIntegrityViolationException to keep database consistent. To achive that without changing existing code, we need to annotate deleteStudentWithLessons(Long id) method with @Transactional annotation. With this spring will update the database states of child entities. In this case it will update Lesson attributes to have student as null. Then student will be removed in the next line.

After this there is a inProgress check for existing lessons. it will throw SQLException which is a checked exception. @Transactional annotation in default only rolls back for unchecked exception such as RuntimeException. So all changes will be reflected to database.

Option A and E is right. More info can be found in https://docs.spring.io/spring-framework/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html

Candidate's Solution

Options: (Expected answer indicated with a tick)

- [x] ** It will try to delete the student then the DataIntegrityViolationException will be thrown. **

- [ ] It will delete the student successfully then an SQLException will be thrown with the message "Cannot delete all Lessons. There are lessons in progress state".

- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException and all database changes will be rolled back.

- [ ] It will throw a RuntimeException with the message "Student not found".

- [ ] If @Transactional annotation is put in place of X (StudentServer.java), after saving the student and lessons, it will update lessons and remove the student. Then it will throw an SQLException.

No comments.

5. Configuration Properties

Incorrect

Multiple Choice | Spring | Spring Boot | Application Properties Annotations | Medium

Question description

An *application.properties* file contains the following properties:

```
app.prop1 = value1
app.prop2 = value2
```

You decided to create a class to hold this configuration. To protect it, the class is defined as final.

```
@Configuration
@ConfigurationProperties(prefix = "app")
public final class AppConfiguration {
  String prop1;
  String prop2;

  public String getProp1() {
    return prop1;
  }

  public void setProp1(String prop1) {
    this.prop1 = prop1;
  }

  public String getProp2() {
    return prop2;
  }

  public void setProp2(String prop2) {
    this.prop2 = prop2;
  }
}
```

What is the result of running the Spring application with this class?

Interviewer guidelines

Spring creates dynamic proxies for classes annotated with `@Configuration` classes. Spring uses CGLIB to extend your class to create proxy. Hence, configuration classes cannot be final. When running the code Spring throws `BeanDefinitionParsingException` and this is the correct option.

Candidate's Solution

Options: (Expected answer indicated with a tick)

- ✓ The application runs, the configuration bean is created. and the properties are initialized correctly (prop1=value1, prop2=value2).

- The application runs, and a configuration bean is created, but properties are set to null (prop1=null, prop2=null).
- A BeanDefinitionParsingException is thrown because the @Configuration bean cannot be marked as final.
- An IllegalStateException is thrown because the @Configuration bean cannot have an @ConfigurationProperties annotation.
- A NoSuchMethodException is thrown because the @Configuration bean needs a default constructor.

No comments.

6. Quiz Competition

Correct

Coding Medium Pointers

Question description

A class has students with various talents, each represented by an integer from 1 to *talentsCount*. You need to form teams for a quiz competition, where each team must have at least one member with each talent.

Teams must be formed from consecutive students in the array. For each possible starting position, determine the minimum number of students needed to form a valid team. If it is not possible to form a team with all talents from a particular starting position, return -1 for that position.

Example

talentsCount = 3

talent = [1, 2, 3, 2, 1]

- Starting at position 1: [1, 2, 3] includes all talents, minimum size = 3
- Starting at position 2: [2, 3, 2, 1] is the smallest subarray with all talents, minimum size = 4
- Starting at position 3: [3, 2, 1] includes all talents, minimum size = 3
- Starting at positions 4 and 5: Cannot form a team with all talents, return -1

The result is [3, 4, 3, -1, -1].

Function Description

Complete the function *teamSize* in the editor with the following parameter(s):

int talent[n]: the students' talents

int talentsCount : the number of talents represented numbered from 1 to *talentsCount*

Returns

int[n]: at each index, the minimum size subarray required, or -1 if an appropriate subarray does not exist

Constraints

- $1 \le n, talentsCount \le 10^5$
- $1 \le talent[i] \le talentsCount$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of elements in *talent*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *talent[i]*.

The next line contains the integer *talentCount*.

▼ SAMPLE CASE 0

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
8       talent[] size n = 8
```

```
1       talent = [1, 1, 2, 2, 3, 1, 3, 2]
1
2
2
3
1
3
2
3       talentsCount = 3
```

Sample Output

```
5
4
4
3
4
3
-1
-1
```

Explanation

The shortest subarrays for each position:

Beginning at the first position: [1, 1, 2, 2, 3] (length 5)

Second position: [1, 2, 2, 3] (length 4)

Third position: [2, 2, 3, 1] (length 4)

Fourth position: [2, 3, 1] (length 3)

Fifth position: [3, 1, 3, 2] (length 4)

Sixth position: [1, 3, 2] (length 3)

No further subarrays will have 3 elements.

▼ SAMPLE CASE 1

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
5       talent[] size n = 5
1       talent = [1, 1, 1, 1, 1]
1
1
1
```

```
1
1       talentsCount = 1
```

Sample Output

```
1
1
1
1
1
```

Explanation

Since there is only 1 type of talent, any 1 person can form a team.

▼ SAMPLE CASE 2

Sample Input For Custom Testing

```
STDIN   Function
-----   --------
9       talent[] size n = 9
7       talent = [7, 5, 3, 4, 6, 1, 7, 2, 4]
5
3
4
6
1
7
2
4
7       talentsCount  7
```

Sample Output

```
8
7
-1
-1
-1
-1
-1
-1
-1
```

Explanation

There have to be at least *7* elements in any subarray to have one from each talent. The shortest subarrays at each position are:

Beginning at the first position: [7, 5, 3, 4, 6, 1, 7, 2] (length 8)
Second position: [5, 3, 4, 6, 1, 7, 2] (length 7)

No further subarrays will have 7 elements.

Interviewer guidelines

▼ SOLUTION

This can be solved using two-pointer technique. For a particular left index (L), we move the right index (R) forward until we have k distinct elements, then once we get it, we can move L forward. Each element will be traversed at most two times.

Time Complexity: O(n) or O(nLogn) (with set/map).

C++ Solution:

```
int f[100005];

vector<int> teamSize(vector<int> talent, int talentsCount) {
    int k = talentsCount;
    int n = talent.size();
    assert(n>=1&&n<=100000);
    assert(k>=1&&k<=n);
    for(int i=0;i<n;i++)
    {
        assert(talent[i]<=k&&talent[i]>=1);
    }
    int cnt = 0;
    int j = 0;
    vector<int> ans(n);
    for(int i=0;i<n;i++)
    {
        int x = talent[i];
        f[x]++;
        if(f[x]==1)
            cnt++;
        while(cnt==k&&j<=i)
        {
            ans[j] = i - j + 1;
            int y = talent[j];
```

```
            f[y]--;
            if(f[y]==0)
                cnt--;
            j++;
        }
    }
    while(j<n)
        ans[j++] = -1;
    return ans;
}
```

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'teamSize' function below.
     *
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY talent
     *  2. INTEGER talentsCount
     */

    public static List<Integer> teamSize(List<Integer> talent, int talentsCount) {
    // Write your code here
        int n = talent.size();
```

```
        int[] freq = new int[talentsCount + 1];
        List<Integer> result = new ArrayList<>();
        int distinct = 0;
        int right = 0;
        for(int left = 0; left < n; left++) {
            while(right < n && distinct < talentsCount) {
                int t = talent.get(right);
                if(freq[t] == 0) distinct++;
                freq[t]++;
                right++;
            }
            if(distinct == talentsCount) {
                result.add(right - left);
            } else {
                result.add(-1);
            }
            int leftTalent = talent.get(left);
            freq[leftTalent]--;
            if(freq[leftTalent] == 0) distinct--;
        }
        return result;
    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int talentCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> talent = IntStream.range(0, talentCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

```

```
        int talentsCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> result = Result.teamSize(talent, talentsCount);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
TestCase 0	Easy	Sample	Success	1	0.067 sec	37.5 KB
TestCase 1	Easy	Sample	Success	1	0.0635 sec	37.8 KB
TestCase 2	Easy	Sample	Success	1	0.0523 sec	37.7 KB
TestCase 3	Easy	Hidden	Success	3	0.0973 sec	49 KB
TestCase 4	Easy	Hidden	Success	3	0.117 sec	49.3 KB
TestCase 5	Easy	Hidden	Success	3	0.1276 sec	47.5 KB
TestCase 6	Easy	Hidden	Success	8	0.3226 sec	70.3 KB

TestCase 7	Easy	Hidden	Success	8	0.3702 sec	70.8 KB
TestCase 8	Easy	Hidden	Success	8	0.3593 sec	71.3 KB
TestCase 9	Easy	Hidden	Success	8	0.3735 sec	70.4 KB
TestCase 10	Easy	Hidden	Success	8	0.3348 sec	74.8 KB
TestCase 11	Easy	Hidden	Success	8	0.3739 sec	71.5 KB
TestCase 12	Easy	Hidden	Success	8	0.3364 sec	70.6 KB
TestCase 13	Easy	Hidden	Success	7	0.2679 sec	68.9 KB

No comments.

7. Java: Optimized Sequential Video Watching

Correct

Coding Medium Greedy Java

Question description

A learner is given a sequential playlist of *n* videos to enhance their knowledge. Implement a function to calculate the minimum time required to complete a total of *m* viewings while adhering to the platform's constraints.

System Constraints

- Sequential Viewing:
  - The learner must watch videos in the order they appear in the playlist.
  - To watch a video at index *i*, the learner must first watch all videos from index 0 to *i-1* at least once.
- Initial Full Viewing:
  - To view any video at index *i* for the first time, the learner must spend *firstWatch[i] + repeatWatch[i]* minutes. This ensures a complete understanding of the video.
- Subsequent Viewings:
  - For subsequent viewings of a video at index *i*, the learner spends only *repeatWatch[i]* minutes.
  - At any time, they can rewatch any video they have already watched.
- Minimum Total Watch Count:
  - The learner must achieve at least *m* total viewings across all videos.
  - Videos can be rewatched as desired, and not all videos need to be watched equally.

Return the minimum time required to complete *m* viewings of videos while following these constraints.

Example

If *n* = 4, *m* = 4, *firstWatch* = [1, 5, 9, 11], and *repeatWatch* = [2, 7, 10, 11], the optimal strategy is as follows:

- The learner watches the 0th video for the first time, spending *firstWatch*[0] + *repeatWatch*[0] = 1 + 2 = 3 minutes.
- The learner then rewatches the 0th video 3 more times in, spending *repeatWatch*[0]*3 = 2 * 3 = 6 minutes.
- The learner has successfully completed 4 viewings. Thus, the total time required is 3 + 6 = 9 minutes, and 9 is returned.

Function Description

Complete the function *findMinMinutes* in the editor below.

Function Parameters

int m: the total number of video viewings to complete

int firstWatch[n]: the minutes required to watch each video the first time

int repeatWatch[n]: the minutes required to rewatch each video

Returns

long: the minimum minutes required for the learner to reach their target number of viewings

Constraints

- $1 \le n \le 10^5$
- $1 \le m \le 10^9$
- $1 \le firstWatch[i], repeatWatch[i] \le 10^9$

INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *m*.
The next line contains an integer, *n,* the size of *firstWatch*.
Each of the next *n* lines contains an integer, *firstWatch[i]*.
The next line contains an integer, *n,* the size of *repeatWatch*.
Each of the next *n* lines contains an integer, *repeatWatch[i]*.

SAMPLE CASE 0

Sample Input 0

```
STDIN    FUNCTION
-----    --------
5        m = 5
3        firstWatch[] size n = 3
6        firstWatch = [6, 8, 9]
8
9
3        repeatWatch[] size n = 3
4        repeatWatch = [4, 2, 10]
2
10
```

Sample Output 0

```
26
```

Explanation

The goal is to complete 5 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time, so the time required is *firstWatch*[0] + *repeatWatch*[0] = 6 + 4 = 10 minutes.
- The learner watches the 1st video for the first time, so the time required is *firstWatch*[1] + *repeatWatch*[1] = 8 + 2 = 10 minutes.
- The learner then rewatches the 1st video 3 times. So the time taken = *repeatWatch*[1]*3 = 2 * 3 = 6 minutes.

- The total time required to complete 5 video viewings is 10 + 10 + 6 = 26 minutes.

▼ SAMPLE CASE 1

Sample Input 1

```
STDIN     FUNCTION
-----     --------
6         m = 6
5         firstWatch[] size n = 5
2         firstWatch = [2, 3, 7, 8, 4]
3
7
8
4
5         repeatWatch[] size n = 5
3         repeatWatch = [3, 6, 8, 7, 6]
6
8
7
6
```

Sample Output 1

```
20
```

Explanation

The goal is to complete 6 viewings. One optimal strategy is:

- The learner watches the 0th video for the first time in *firstWatch*[0] + *repeatWatch[0]* = 2 + 3 = 5 minutes.
- The learner then rewatches the 0th video 5 times in *repeatWatch[0]**5 = 3 * 5 = 15 minutes.
- The total time required to complete 6 viewings is 5 + 15 = 20 minutes.

Interviewer guidelines

▼ SOLUTION

Description

The greedy solution suggests that, at any given index, if the time to watch the video is less than at any other index between i+1 and n, the minimum total time will be the sum of the watch time up to index i-1, plus the repeat time of the ith video multiplied by (m-i).

Solution

```
class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
        long minMinutes = Long.MAX_VALUE;
        long cur = 0;
        int n = firstWatch.size();

        for (int i = 0; i < Math.min(n,m); i++) {
            int a = firstWatch.get(i);
            int b = repeatWatch.get(i);

            minMinutes = Math.min(minMinutes, cur + ((long) (m - i) * b) + a);

            cur += a + b;
        }
        return minMinutes;
    }

}
```

▼ COMPLEXITY

Time Complexity - O(n)

The algorithm iterates through each number in *firstWatch* and *repeatWatch* exactly once, performing constant-time checks and arithmetic operations in each iteration. Thus, the time complexity is O(n), where *n* is the length of *firstWatch*.

Space Complexity - O(1)

Space usage is constant, only utilizing a fixed number of variables (*mn, cur,* etc.) that do not scale with the length of *firstWatch*. Therefore, the space complexity is O(1).

Candidate's Solution

Language used: **Java 21**

```java
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'findMinMinutes' function below.
     *
     * The function is expected to return a LONG_INTEGER.
     * The function accepts following parameters:
     *  1. INTEGER m
     *  2. INTEGER_ARRAY firstWatch
     *  3. INTEGER_ARRAY repeatWatch
     */

    public static long findMinMinutes(int m, List<Integer> firstWatch, List<Integer> repeatWatch) {
    // Write your code here
        int n = firstWatch.size();
        long time = 0;
        long ans = Long.MAX_VALUE;

        PriorityQueue<Integer> minRepeat = new PriorityQueue<>();
        for(int i = 0; i < n && i < m; i++) {
            time += firstWatch.get(i) + repeatWatch.get(i);
            minRepeat.offer(repeatWatch.get(i));

            long viewsDone = i + 1;
            long remaining = m - viewsDone;

            if(remaining > 0)
                time += remaining * minRepeat.peek();
            ans = Math.min(ans, time);
```

```
            if(remaining > 0) {
                time -= remaining * minRepeat.peek();
            }
        }
        return ans;

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int m = Integer.parseInt(bufferedReader.readLine().trim());

        int firstWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> firstWatch = IntStream.range(0, firstWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int repeatWatchCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> repeatWatch = IntStream.range(0, repeatWatchCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
```

```
            .map(Integer::parseInt)
            .collect(toList());

        long result = Result.findMinMinutes(m, firstWatch, repeatWatch);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

TESTCASE	DIFFICULTY	TYPE	STATUS	SCORE	TIME TAKEN	MEMORY USED
Testcase 2	Easy	Sample	Success	1	0.0649 sec	37.5 KB
Testcase 0	Easy	Sample	Success	1	0.0573 sec	37.4 KB
Testcase 1	Easy	Sample	Success	1	0.0581 sec	37.6 KB
Testcase 3	Easy	Hidden	Success	4	0.0522 sec	37.5 KB
Testcase 4	Easy	Hidden	Success	4	0.1025 sec	38.1 KB
Testcase 5	Easy	Hidden	Success	4	0.0942 sec	46.3 KB
Testcase 6	Medium	Hidden	Success	5	0.3343 sec	69.4 KB
Testcase 7	Medium	Hidden	Success	5	0.6275 sec	171 KB

Testcase 8	Medium	Hidden	Success	5	0.6142 sec	177 KB
Testcase 9	Medium	Hidden	Success	5	0.5421 sec	170 KB
Testcase 10	Medium	Hidden	Success	5	0.6589 sec	153 KB
Testcase 11	Medium	Hidden	Success	5	0.5368 sec	157 KB
Testcase 12	Hard	Hidden	Success	10	0.3565 sec	74.8 KB
Testcase 13	Hard	Hidden	Success	10	0.6038 sec	120 KB
Testcase 14	Hard	Hidden	Success	10	0.0618 sec	37.6 KB

No comments.